



Last updated on **Mar 31, 2026**

What is Delta Lake in Databricks?

Delta Lake is the optimized storage layer that provides the foundation for tables in a lakehouse on Databricks. Delta Lake is [open source software](#) that extends Parquet data files with a file-based transaction log for [ACID transactions](#) and scalable metadata handling. Delta Lake is fully compatible with Apache Spark APIs, and was developed for tight integration with Structured Streaming, allowing you to easily use a single copy of data for both batch and streaming operations and providing incremental processing at scale.

Delta Lake is the default format for all operations on Databricks. Unless otherwise specified, all tables on Databricks are Delta tables. Databricks originally developed the Delta Lake protocol and continues to actively contribute to the open source project. Many of the optimizations and products in the Databricks platform build upon the guarantees provided by Apache Spark and Delta Lake. For information on optimizations on Databricks, see [Optimization recommendations on Databricks](#).

For reference information on Delta Lake SQL commands, see [Delta Lake statements](#).

The Delta Lake transaction log has a well-defined open protocol that can be used by any system to read the log. See [Delta Transaction Log Protocol](#).

Getting started with Delta Lake

All tables on Databricks are Delta tables by default. Whether you're using Apache Spark [DataFrames](#) or SQL, you get all the benefits of Delta Lake just by saving your data to the lakehouse with default settings.

For examples of basic Delta Lake operations such as creating tables, reading, writing, and updating data, see [Tutorial: Create and manage Delta Lake tables](#).

Databricks has many recommendations for [best practices for Delta Lake](#).



Converting and ingesting data to Delta Lake

Databricks provides a number of products to accelerate and simplify loading data to your lakehouse.

- Lakeflow Spark Declarative Pipelines:
 - [Get started: Build an extract, transform, and load \(ETL\) pipeline in Databricks](#)
 - [Load data using streaming tables \(Python/SQL notebook\)](#)
 - [Streaming tables](#)
- [COPY INTO](#)
- [Auto Loader](#)
- [Add data UI](#)
- [Incrementally convert Parquet or Iceberg data to Delta Lake](#)
- [One-time conversion of Parquet or Iceberg data to Delta Lake](#)
- [Third-party partners](#)

For a full list of ingestion options, see [Standard connectors in Lakeflow Connect](#).

Updating and modifying Delta Lake tables

Atomic transactions with Delta Lake provide many options for updating data and metadata. Databricks recommends you avoid interacting directly with data and transaction log files in Delta Lake file directories to avoid corrupting your tables.

- Delta Lake supports upserts using the merge operation. See [Upsert into a Delta Lake table using merge](#).
- Delta Lake provides numerous options for selective overwrites based on filters and partitions. See [Selectively overwrite data with Delta Lake](#).
- You can manually or automatically update your table schema without rewriting data. See [Update table schema](#).

 Ask Genie

- Enable columns mapping to rename or delete columns without rewriting data. See [Rename and drop columns with Delta Lake column mapping](#).

Incremental and streaming workloads on Delta Lake

Delta Lake is optimized for Structured Streaming on Databricks. [Lakeflow Spark Declarative Pipelines](#) extends native capabilities with simplified infrastructure deployment, enhanced scaling, and managed data dependencies.

- [Delta table streaming reads and writes](#)
- [Use Delta Lake change data feed on Databricks](#)

Querying previous versions of a table

Each write to a Delta table creates a new table version. You can use the transaction log to review modifications to your table and query previous table versions. See [Work with table history](#).

Delta Lake schema enhancements

Delta Lake validates schema on write, ensuring that all data written to a table matches the requirements you've set.

- [Schema enforcement](#)
- [Constraints on Databricks](#)
- [Delta Lake generated columns](#)
- [Enrich tables with custom metadata](#)

Managing files and indexing data with Delta Lake

Databricks sets many default parameters for Delta Lake that impact the size of data files and number of table versions that are retained in history. Delta Lake uses a combination of  Ask Genie

metadata parsing and physical data layout to reduce the number of files scanned to fulfill any query.

- [Use liquid clustering for tables](#)
- [Data skipping](#)
- [Optimize data file layout](#)
- [Remove unused data files with vacuum](#)
- [Control data file size](#)

Configuring and reviewing Delta Lake settings

Databricks stores all data and metadata for Delta Lake tables in cloud object storage. Many configurations can be set at either the table level or within the Spark session. You can review the details of the Delta table to discover what options are configured.

- [Review table details with describe detail](#)
- [Table properties reference](#)

Data pipelines using Delta Lake and Lakeflow Spark Declarative Pipelines

Databricks encourages users to leverage a [medallion architecture](#) to process data through a series of tables as data is cleaned and enriched. [Lakeflow Spark Declarative Pipelines](#) simplifies ETL workloads through optimized execution and automated infrastructure deployment and scaling.

Delta Lake feature compatibility

Not all Delta Lake features are in all versions of Databricks Runtime. For information about Delta Lake versioning, see [Delta Lake feature compatibility and protocols](#).

Delta Lake API documentation

For most read and write operations on Delta tables, you can use [Spark SQL](#) or Apache Spark [DataFrame](#) APIs.

For Delta Lake-specific SQL statements, see [Delta Lake statements](#).

Databricks ensures binary compatibility with Delta Lake APIs in Databricks Runtime. To view the Delta Lake API version packaged in each Databricks Runtime version, see the **System environment** section on the relevant article in the [Databricks Runtime release notes](#). For documentation on Delta Lake APIs for Python, Scala, and Java, see the [OSS Delta Lake documentation](#).

On this page

Was this page helpful?